What Does Spiking Cost an Autoencoder? A Group-Disjoint Comparison with a Non-Spiking Control on Speech and EEG

André Furlan
Instituto de Biociências, Letras e Ciências Exatas – UNESP
andre.furlan@unesp.br

***Abstract*—Spiking autoencoders have been compared with non-spiking networks of matching size mostly on images. We ask how much reconstruction quality a selected spiking pipeline costs, and what it returns in energy, for short speech and single-channel EEG windows. The spiking autoencoder (SNN-AE) is compared with PCA and with a control (ANN-AE) that copies its selected widths, initializer and training, replaces the leaky integrate-and-fire neuron by a leaky ReLU, and reads the analog window once, without a spike code, input transform or time axis. The SNN-AE selects its architecture, the number of simulation steps, the initial threshold and decay, the spike encoding and an input transform by multi-objective search inside nested group-disjoint 6-fold cross-validation on four datasets, and differences are tested across held-out speakers and subjects. The target is a low-power device, so energy is estimated per window with an analytical model after Lemaire et al. that counts memory accesses and address arithmetic as well as operations, at published 45 nm prices, for the whole model and for its encoder alone. Every energy comparison is repeated under three memory sizes, at half the memory price, with arithmetic alone, and with a direct-coded input's first layer recomputed at every step. Searched LSTM, GRU and Transformer autoencoders, which differ from the SNN-AE in more than the neuron, are reported under one reduced search budget on development folds, for their training and inference cost and their quality, and left out of the main comparison. A design study on those folds examines three choices the adopted design inherited: the loss, the decoder and the initializer. Those folds, and the fold that tests on their validation groups, are kept out of the headline numbers.**

***Index Terms*—spiking neural networks, autoencoder, non-spiking control, energy estimate, nested cross-validation, speech, EEG**

I. Introduction

Spiking neural networks (SNNs) communicate with binary events. A synapse whose input did not spike does no work, which saves energy only when the input is sparse, and a synapse whose input did spike adds a weight instead of multiplying by it [1]. Surrogate gradients [2]–[4] made such networks trainable end to end, and autoencoders are an obvious use: compress a signal window to a small code on a low-power device and reconstruct it elsewhere.

Whether that trade is worth it is a question about two numbers measured on networks of the same widths: how much reconstruction error a spiking pipeline adds, and how much energy it saves. The second must count what spiking adds as well as what it removes: a spike turns a multiply–accumulate into an add, but every operand must also be fetched, at a higher price than the arithmetic, and a spiking layer reads and writes its membrane potentials at every simulation step [1]. Published comparisons answer a different question. Spiking autoencoders have been reported as comparable to conventional ones on MNIST-type images, against baselines of similar size [5] or the same architecture [6]; when the comparison is against recurrent or attention models, those differ in recurrence, sequence length and capacity at once, so a quality difference cannot be attributed to the neuron. We are not aware of a comparison against a non-spiking network of the same widths for the reconstruction of speech or EEG.

Two further confounds are easy to miss. First, evaluation: windows of one recording and one speaker are near-duplicates, so pooling windows before splitting measures memorization, and selecting hyperparameters on the test speaker biases every number [7]. Second, design choices that are not the spike can skew the comparison: Section V examines three of them, a per-step loss, a spiking decoder and the initializer.

This paper makes four contributions.

1) A *non-spiking control*: the ANN-AE copies the selected SNN-AE's widths, initializer, optimizer and data, and differs from it in the encoder's hidden neuron and in everything spiking requires around it (the time axis, the spike code and the input transform), so the gap between them is the cost of the selected spiking pipeline.
2) A group-disjoint protocol: nested 6-fold cross-validation (leave-one-speaker-out on FSDD) on two speech and two EEG datasets, with the SNN-AE's architecture selected on inner validation groups only, and paired statistics across held-out groups.
3) A quality–energy accounting that counts memory accesses and address arithmetic as well as operations, after the analytical model of [1] at the 45 nm prices of [8], applied alike to every model and to PCA, the SNN's spike counts taken from its executed forward pass, and every comparison checked under three memory sizes, at half the memory price, under arithmetic alone, and with a repeated input recomputed at every step.
4) A design study, on development folds kept out of the headline numbers, of three choices that are not the spike, and a pre-validation on the same folds that reports what

searched sequence autoencoders cost to train and to run.

II. Related Work

Spiking autoencoders. Comşa et al. [5] trained spiking autoencoders with temporal coding and found reconstruction errors comparable to conventional autoencoders of similar size on MNIST and Fashion-MNIST. The fully spiking VAE [6] compared against a non-spiking VAE of the same architecture on image datasets. Neither addresses temporal signals or a group-disjoint protocol.

Training and coding. We train with surrogate gradients and backpropagation through time [2], [9], [10], and use the three common input codes: direct currents, rate (Bernoulli) coding, and latency coding [11], [12].

Energy accounting. A widespread convention prices an SNN's analog-input layers by multiply–accumulates (MACs) and its spike-driven layers by accumulations (ACs), with 45 nm figures from Horowitz [8]; Spikformer [13] is one example. It leaves out memory, which in the same table costs more than the arithmetic it feeds (a 32-bit float add 0.9 pJ, one access to a 32 KB SRAM 20 pJ), and with it the traffic only an SNN has: its potentials, read and written at every step. Estimates that count memory do not agree on the outcome, which depends on the spike rate, the number of steps and the hardware: Lemaire et al. [1] find their SNNs 6 to 8 times more efficient than non-spiking networks, Dampfhoffer et al. [14] that SNNs match efficient ANN accelerators only at very high sparsity, and Yan et al. [15] that an advantage requires strict limits on both. We therefore count memory, and report whether each conclusion survives a change of the memory model (Section III-G).

Sequence autoencoders and protocol. LSTM [16], [17], GRU [18], [19] and self-attention [20], [21] autoencoders are the usual non-spiking baselines for sequences; because they differ from an SNN-AE in more than the neuron, we report them in a pre-validation only (Section V-A). Nested evaluation is the standard remedy for selection bias in model selection [7]; we also keep groups disjoint across splits.

III. Methods

A. Task

A window $x \in \mathbb{R}^M$ of $M = 256$ consecutive samples is z-scored on its own statistics and reconstructed through a latent $z \in \mathbb{R}^d$, minimizing the mean squared error. The latent width d is fixed per signal domain ($d = 16$ for speech, $d = 64$ for EEG) and is never searched, so every model faces the same compression ratio. A recording's last window is zero-padded; padded samples are excluded from normalization, every loss and every reported error.

B. SNN-AE

Neuron. The encoder's hidden layers are leaky integrate-and-fire (LIF) neurons [22] simulated for T steps:

$$V[t] = \beta\, V[t-1] + I[t], \qquad s[t] = \mathbf{1}[V[t] > V_{th}], \qquad (1)$$

and $V[t] \leftarrow 0$ after a spike (hard reset), with $\beta = e^{-\Delta t/(RC)}$ and $\Delta t = 1$. R, C and V_{th} are trained per layer (R and C clamped above 10^{-6}; nothing bounds V_{th}, and its trained values are in Table VII) from initial values $R = 1$, $C = -1/\ln\beta^{(0)}$ and $V_{th}^{(0)}$, so the searched $\beta^{(0)}$ is the initial decay. The spike's zero gradient is replaced in the backward pass by the arctan surrogate [9] $\hat{\sigma}'(V) = \alpha/\big(2(1 + (\frac{\pi}{2}\alpha(V - V_{th}))^2)\big)$, $\alpha = 2$, and the error is propagated through time as

$$\delta_l[t] = \hat{\sigma}'(V_l[t]) \odot \big(W_{l+1}^{\top}\delta_{l+1}[t]\big) + \beta\,(1 - s_l[t]) \odot \delta_l[t+1], \quad (2)$$

where the reset is detached, so no error flows through a step that fired. The gradient of V_{th} keeps the surrogate through the reset, a term summed once and not multiplied through time.

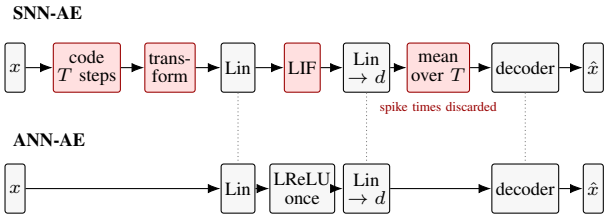


Figure 1. The SNN-AE and its control. Shared blocks have identical widths, initializer and training. Red: the parts that exist only in the spiking network, which is what the gap measures. The mean over T discards when each neuron fired.

Network. The encoder is $\mathrm{Lin}(M \to w_1) \to \mathrm{LIF} \to \cdots \to \mathrm{Lin}(w_{L-1} \to d)$ with $w_1 > \cdots > w_{L-1} > d$; the latent layer is analog. The latent is averaged over the T steps and decoded *once* by the mirror image of the encoder with leaky ReLUs (slope 0.01) and a linear output (Fig. 1). The loss is computed on that single reconstruction; Section V compares this with an earlier design that scored the reconstruction of every step and spiked in the decoder too.

Input. The window is presented for T steps as a constant current (*direct*), as Bernoulli spikes with probability proportional to the min–max-scaled sample (*Poisson*), or as one spike per sample at a time that decreases with its value (*latency*). An optional input transform is applied before the network: none (*dense*), a fixed 3-tap smoothing along the window (*conv1d*), or one LIF cell per sample with the searched initial threshold and decay, untrained and with a soft reset (*recurrent*), which turns any code into spikes.

C. Control (ANN-AE)

For every dataset, fold and run, the ANN-AE takes the widths of that run's selected SNN-AE and replaces each hidden LIF by a leaky ReLU applied once to the analog window; it is trained with the same initializer, optimizer, data and stopping rule. It is *not* searched, so the SNN-AE receives the larger search budget: a gap in the ANN-AE's favour understates what a searched control could reach. The control is not otherwise matched to the code, T or transform the search selects.

D. References

PCA with $k = d$ components, fitted on the fold's capped training and validation windows, is the best linear code through the same bottleneck, and is priced like the networks

(Section III-G); a mean-window predictor fixes the scale of "no model".

E. Architecture search

The SNN-AE's architecture is selected by NSGA-II [23] with population 8 for 5 generations (48 candidates), minimizing validation MSE and an analytic inference cost. The final front is re-scored over 3 fresh seeds, the search's own score left out, and the member with the lowest re-scored validation MSE wins (ties: lower cost). Each dataset and fold draws its own candidates; the trainings that score them use the run's seed. The genes are the number of hidden layers, $L-1 \in [1,3]$; their widths, in $(d, 128]$; $T \in [2,32]$ (an integer drawn uniformly); $V_{th}^{(0)} \in [0.5, 1.5]$; $\beta^{(0)} \in [0.8, 0.99]$; the encoding; and the input transform. A width is drawn from $[4, 128]$ and raised to $d+1$ when it falls below, which on EEGMMIDB happens to 50% of the draws; Section V tests wider controls. The cost objective is a spike-blind MAC count in which T multiplies every encoder layer that runs at every step (a direct-coded first layer counts once, as in Section III-G). It keeps cheaper genomes on the front, but the winner is chosen by validation MSE alone, so a longer simulation is selected only when it validates better.

F. Training

All models use Adam [24] (learning rate 10^{-3}; 10^{-4} for R, C, V_{th}), batch size 1 and at most 60 epochs. Early stopping [25] has patience 5, counted only once the monitored loss is 10% below that of the mean predictor (the mean training window, scored on the same monitor windows). On z-scored windows a network can start on that plateau. In an earlier run of the pre-validation (Section V-A) plain patience stopped networks that had not left it yet, and they entered the comparison as the mean predictor under another name. The level was set on the monitored loss curves of the development folds; no test number enters it. A pre-set automatic rule did not endorse it, because some sequence models learn a little without reaching it; we kept it because replaying the curves showed that a lower level saves under 1% of the epochs. A run that never reaches it trains all 60 epochs, and every run keeps the weights of its best monitored epoch [26, Alg. 7.1]. The SNN-AE and the ANN-AE are both initialized with Xavier [27]. Left to their builders, the two arms had different defaults (fan-in scaling [28] for the SNN); Section V examines what that does, so the initializer is fixed and shared. Within a fold, the selected architecture is retrained on the training and validation groups, except for a recording-disjoint slice of the training groups that is held out for early stopping, and evaluated once on the test group. Each fold is repeated 5 times (seeds 42–46), each repeat with its own search.

G. Energy

E is the energy of one window's forward pass, after training. It must count what spiking adds as well as what it removes, so it follows the analytical model of Lemaire et al. [1], extended as listed below and applied alike to every model:

$$E = E_{\text{ops}} + E_{\text{addr}} + E_{\text{mem}}, \tag{3}$$

the arithmetic, the integer additions that compute memory addresses, and the memory accesses, with the counts of Table I. The prices are Horowitz's for 45 nm at 0.9 V [8]: 32-bit float add 0.9 pJ and multiply 3.7 pJ (a MAC 4.6 pJ), 32-bit integer add 0.1 pJ, and one SRAM access 10, 20 and 100 pJ at 8 KB, 32 KB and 1 MB. These are Horowitz's figures for a 64-bit cache access; we apply them to one 32-bit word, which overprices every access, so Table IV also halves them. Lemaire et al. price operations with 32-bit integer figures; we use the floating-point ones, since the weights are 32-bit floats. As in [1], each layer holds its weights, its potentials and its input and output buffers in local SRAMs, each priced at its own size on the straight lines through those three points, and one access moves one 32-bit word. The SNN-AE's counts come from its forward pass on each test window, with that window's spikes; the ANN-AE's follow from its architecture. A column reports the encoder alone (spike code, input transform and encoder network), which is what a sensor pays if the code is decoded elsewhere. PCA is priced by the same model as a linear encoder and decoder, its mean included.

The model leaves several choices, and we state the direction of each so that a conclusion can be read against it. Against the SNN-AE: memories at or below 8 KB, its potentials among them, are priced at 8 KB's 10 pJ; the LIF's threshold test costs one add per neuron and step, which [1] do not count; the spike encoder is charged to it (Poisson: a pseudo-random draw and a comparison per sample and step; latency: one spike time per sample and a comparison per sample and step), and the ANN-AE has none. In its favour: leaky ReLU costs a comparison and a multiply; and direct coding presents the same window at every step, so the first layer (and the smoothing filter of the conv1d input transform in front of it) computes the same current T times, while E counts it once per window, plus one write of that current and its read at every step, as an accelerator that reuses a constant input would. Reading the window from off-chip memory is left out for every model. Because estimates of this kind turn on the memory model [14], [15], every energy comparison is repeated with every access priced at one SRAM size (8 KB, 32 KB, 1 MB), with every memory price halved, with arithmetic alone ($4.6\,\text{pJ} \cdot N_{\text{MAC}} + 0.9\,\text{pJ} \cdot N_{\text{AC}}$, the convention of [13]), and with that first layer recomputed at every step, as our implementation runs it; Table IV reports all of them. E estimates a modeled accelerator; it is not a measurement on a chip.

When can a spike-fed first layer cost less than an analog one at all? Fig. 2 applies these counts to the first stage, from the window to the first hidden code, at the first hidden widths the pre-validation selected (Section V-A). Fed with spikes, that stage costs less than the ANN-AE's first layer (227–459 nJ) only when the input is sparse: fewer spikes per sample and window than a break-even that falls with T. For a Poisson code the break-even runs from 0.61 at $T = 2$ to 0.16 at $T = 32$;

Table I

COUNTS PER LINEAR LAYER $n_{\text{in}} \to n_{\text{out}}$ OVER R ROWS ($R = T$ STEPS FOR THE SNN-AE, BUT 1 FOR THE FIRST LAYER OF A DIRECT-CODED INPUT; 1 FOR THE ANN-AE), AFTER [1]; θ: INPUT SPIKES.

	Analog input	Spike input
MAC	$R\,n_{\text{in}}\,n_{\text{out}}$	–
Add (bias, synapse)	$R\,n_{\text{out}}$	$(\theta + R)\,n_{\text{out}}$
Address add (integer)	$R(n_{\text{in}} + n_{\text{out}})$	$\theta\,n_{\text{out}}$
Read input	$R\,n_{\text{in}}$	θ
Read weight, bias	$R(n_{\text{in}} + 1)\,n_{\text{out}}$	$(\theta + R)\,n_{\text{out}}$
Read, write potential	–	$(\theta + R)\,n_{\text{out}}$ each
Write output	$R\,n_{\text{out}}$	$R\,n_{\text{out}}$ if no LIF follows
Stored first layer (direct)	write n_{out}, read $T\,n_{\text{out}}$	–

LIF, per neuron and step: the leak (MAC) and the threshold test (add), and after an analog-input layer a read and a write of the potential; per output spike: the reset (add) and one write of the event. A layer that feeds an LIF writes no output of its own, except the stored current above. Mean over T of the analog latent: an add and a read per value, a MAC and a write per output.

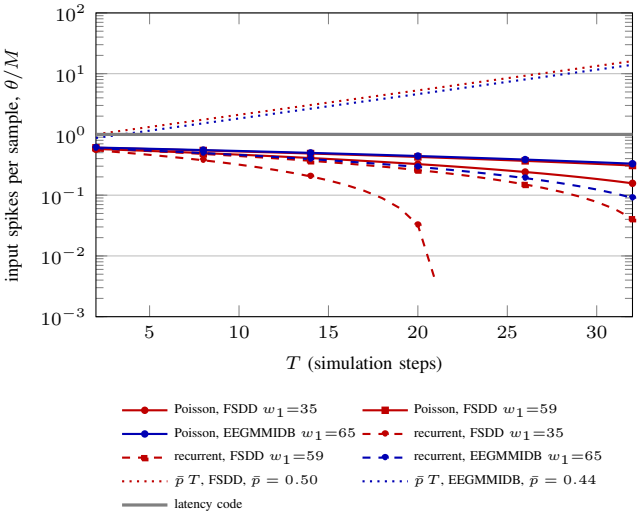


Figure 2. When a spike-fed first stage costs less than the ANN-AE's first layer: below its break-even curve (solid: Poisson code; dashed: direct code through the recurrent transform), at each first hidden width w_1 the pre-validation selected. Dotted: the spikes a Poisson code emits, $\bar{p}\,T$, with $\bar{p}$ measured on the development folds' test windows; grey: a latency code. Logarithmic axis: a curve that ends before $T = 32$ has reached zero, where the stage costs more than the analog layer even without a spike. The model of Table I, without the hidden layer's own spikes.

through the recurrent transform, whose own neurons are priced too, from 0.59 to none at $T = 32$, where for the narrowest winner the transform alone costs more than the analog layer. The codes do not get there. A latency code emits exactly one spike per sample, above every break-even. A Poisson code emits $\bar{p}\,T$ spikes per sample, and because each window is min–max scaled, $\bar{p} = 0.44 - -0.50$ on the development folds: the code already carries 0.87 spikes per sample at $T = 2$ and stays above its break-even at every T. Only the recurrent transform, whose spike count is set by its own threshold and decay, can feed the network sparsely enough. The first hidden layer's own spikes are not charged here, so each break-even is an upper bound.

IV. EXPERIMENTAL SETUP

Datasets (Table II). FSDD [29] and AudioMNIST [30] are spoken digits, the latter resampled offline from 48 to 8 kHz; EEGMMIDB [31], [32] (motor movement/imagery) and Siena [33] (epilepsy) are scalp EEG, read from the first channel of each recording (Fc5 and Fp1 in the file headers), band-passed to 0.5–40 Hz with a mains notch (60 Hz EEGMMIDB, 50 Hz Siena) at each file's own rate, then converted to 160 Hz by band-limited interpolation: the 36 EEGMMIDB files recorded at 128 Hz and every Siena file (512 Hz). A window thus lasts the same time in every EEG recording. At Siena's native rate a window would be too short: it would hold fewer independent values (twice the band times the duration) than its latent has entries, and a linear projection would reconstruct it exactly. The group is the speaker or subject. Windows do not overlap; a long recording keeps a number of windows drawn uniformly across it, and each fold keeps at most 1000 training, 150 validation and 1500 test windows, balanced by speaker or subject, then digit, then recording. The composition of every fold is in the released manifests. The draw is fixed per fold, so every seed sees the same windows and the seeds do not sample it.

Nested group-disjoint cross-validation. Groups are sorted and split into 6 contiguous blocks. In fold f block f is the test set, block $f+1$ (mod 6) the validation set, and the rest train (Fig. 3); for FSDD this is leave-one-speaker-out. Every window is assigned by its group before any pooling, and the harness aborts if a recording or group appears in two splits. All selection uses the validation block only.

Development folds. The design study (Section V) compared variants on fold 0 of FSDD and of EEGMMIDB, including their test windows, and read the errors on that fold's validation block, which is the test block of fold 1. Both folds are

Table II

DATASETS. G: GROUP UNIT; CAP: WINDOWS KEPT PER RECORDING; d: LATENT WIDTH.

Dataset	Signal	G	#G	cap	d
FSDD	speech, 8000 Hz	speaker	6	–	16
AudioMNIST	speech, 8000 Hz	speaker	60	2	16
EEGMMIDB	EEG, 160 Hz	subject	109	40	64
Siena	EEG, 160 Hz	subject	14	40	64

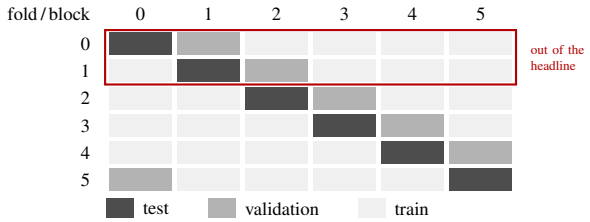


Figure 3. Which block plays which role in each fold, and what the design study touched. Folds 0 and 1 of FSDD and EEGMMIDB leave the headline numbers; fold 5 stays, although it validates on the block the design study tested on.

therefore excluded from every headline number: FSDD and EEGMMIDB are reported on folds 2–5, AudioMNIST and Siena on folds 0–5. Including them is reported as a sensitivity analysis (Table VIII). This does not make those two datasets untouched: fold 5 validates on the development test block, and every FSDD speaker entered the development fold as test, validation or training, so FSDD and EEGMMIDB results are design-informed. AudioMNIST and Siena were not used in the design study. Also chosen on the development folds: the stopping level, the cap of 1000 training windows, the range of T and the surrogate's sharpness.

Statistics. Every model reconstructs the same test windows. A new user is a new speaker or subject, so the primary estimand is the group-level paired difference d_g, the SNN-AE's mean window error on held-out group g minus a reference model's, seeds averaged. We report $\overline{d_g}$, a 95% Student-t interval over groups, and the one-sample t-test p, Holm-corrected across the three references within each dataset [34]; datasets are not pooled or corrected against each other. Each d_g is drawn (Fig. 8). We use t rather than a rank test because with 4 FSDD speakers the exact Wilcoxon p cannot fall below 0.125, and rather than a bootstrap because a percentile interval undercovers at that n. The t interval rests on approximate normality, which 4 points cannot check, and on independent groups, which holds only in part: subjects of one fold share their models, and folds share training groups. We therefore also compute the interval over fold means. Recording-, seed- and window-level results are in the released output and are descriptive: recordings of one subject share an effect, and windows are pseudoreplicated.

Reading the two axes. The application is a low-power device, so we fixed, after seeing the development folds and before the main run, how the two axes are read. For the SNN-AE against the ANN-AE and against PCA: where the whole interval for the mean difference lies inside $\pm\Delta$, with $\Delta = 0.02$, both over groups and over fold means, we recommend the model with the lower E. Otherwise we report what the lower-energy model gives up, its error difference against its energy ratio, instead of ranking on either axis alone. Δ is 2% of the window's variance, a change we treat as negligible, between 0.12 dB at an error of 0.71 and 0.46 dB at 0.18; it is a judgement of relevance, not an estimate of noise. An interval too wide to fit inside $\pm\Delta$ supports no equivalence, whatever its centre: with 4 FSDD speakers it fits only if their standard deviation is below 0.0126, so on FSDD we expect to report the trade. Selection inside the search is unchanged by this rule (Section III-E).

V. Design Study on the Development Folds

Before the main run we asked what the design inherited. Earlier versions of this study ran on windows with four defects that the pipeline no longer has: the 44 bytes of each WAV header were decoded as 22 samples, so window 0 of every speech recording was mostly header; window caps kept a prefix of a sorted list (FSDD fold 0 trained on digits 0–4); a per-recording cap kept each recording's start; and 36 EEGMMIDB files at 128 Hz were treated as 160 Hz. No earlier number is reported here.

The study runs on a PyTorch replica of the network that keeps the harness's code, neuron and gradient, initializers, optimizer, batch size, stopping rule and best-epoch weights. It takes the four SNN-AE winners of the pre-validation (Section V-A; three are Poisson-coded) and trains each with 4 seeds in two crossed blocks (208 trainings in all): the read-out (loss on every step, loss on the time mean, or the latent's time mean) crossed with the decoder's neuron, and the encoder's hidden neuron crossed with the initializer. Seeds pair the cells: a seed fixes the initial weights, the window order and the Poisson draws. Fig. 4 draws every cell and seed, and the harness's own error at the two points both ran; the replica is within 0.034 of them. Intervals below are seed-paired Student-t 95% intervals: they describe training noise on fixed windows, not variation over speakers or subjects.

(i) The loss. Scoring the reconstruction of every simulation step asks each step, carrying at most one binary event per neuron, to reproduce the analog window alone. On the Poisson-coded winners it raised the test error by 0.06 to 0.12 over scoring the time mean, every interval excluding zero; on the direct-coded winner no interval excluded zero. *(ii) The decoder.* A spiking decoder runs T times and leaky ReLUs once, so it is a cost choice in any case; behind the latent's time mean it added up to 0.07, significant on EEGMMIDB only. The adopted design is at most 0.006 above the best cell of block A at every winner. *(iii) The initializer.* Fan-in scaling made the leaky-ReLU control 0.20 worse on EEGMMIDB, enough to reverse the comparison there (SNN-AE 0.43 and 0.46 against 0.50; the difference's interval excludes zero for one of them), and changed the SNN-AE by at most 0.03. Xavier's mean error is the lower for both arms at every winner, which is why they share it: the comparison is not won by handicapping the control, but on EEGMMIDB its sign depends on that choice.

(iv) Bounded codes. With Xavier, the LIF encoder cost 0.04 to 0.06 over the leaky ReLU on FSDD and 0.10 to 0.13 on EEGMMIDB. A sigmoid, bounded but smooth, stayed within 0.012 of the leaky ReLU, no interval excluding zero; a hard clamp to $[0, 1]$ cost 0.05 to 0.09 on FSDD and 0.19 on EEGMMIDB, and an identity gradient through the clamp cost more, so a dead gradient is not what hurts. The direct-coded winner's code is near binary (48% of (neuron, window) pairs never fire and 39% fire at every step), and the reason is simple. Under direct input a neuron sees the same current I at every step, so with a hard reset it never fires for $I \leq V_{th}(1-\beta)$ and fires at every step for $I > V_{th}$. Between them its time mean is a staircase with levels near $1, 1/2, 1/3, \ldots$, and no level between $1/2$ and 1 at any T (Fig. 5a). The Poisson-coded winners' codes are not near binary (25 to 30% silent, at most 3% always firing; Fig. 5b,c), yet their gaps (0.04 to 0.13) are of the same order, so saturation does not account for them. We claim no cause for the gap.

(v) The budget. In this study 58% of the LIF trainings had their best epoch among the last three of the 30, against 38% of the leaky-ReLU ones. Training the adopted SNN-AE and its control for 60 epochs instead of 30 lowered either's error by at most 0.015 and moved their gap by at most 0.009, no interval

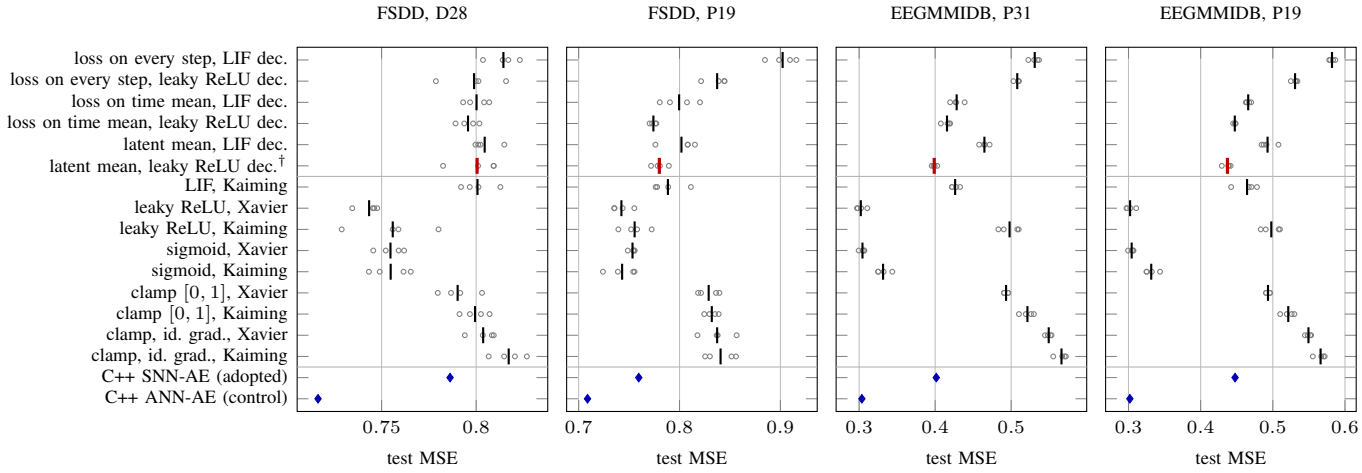


Figure 4. Design study, development folds: test MSE of every cell, one circle per seed (4 seeds) and a bar at their mean; red and †: the adopted design. Panels: the four SNN-AE winners of the pre-validation, by code (D direct, P Poisson) and T. Upper block: read-out × decoder, LIF encoder, Xavier. Middle block: the encoder's hidden neuron × initializer, latent mean, leaky-ReLU decoder (the leaky-ReLU, Xavier row is the control). Diamonds: the C++ harness, one seed. The values are released with the code.

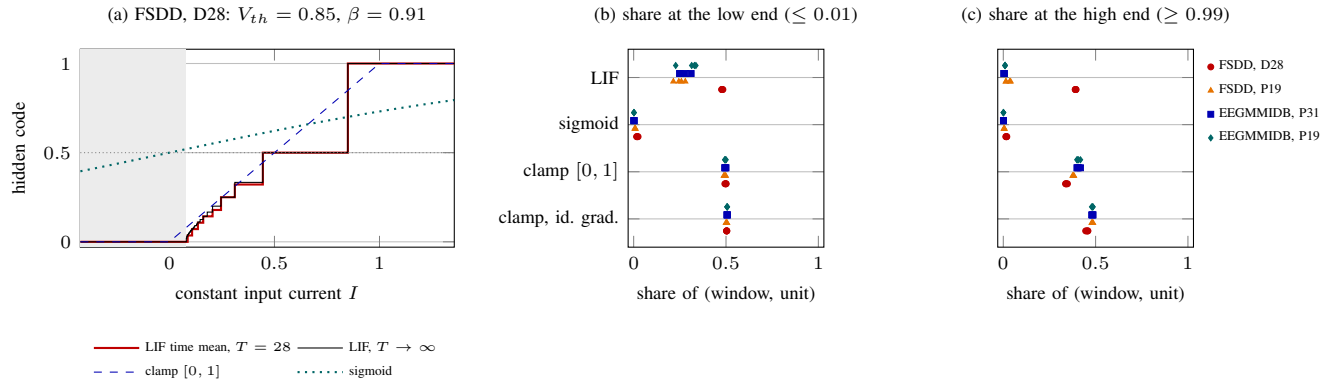


Figure 5. (a) The hidden code of the direct-coded winner as a function of a constant input current: the LIF's time mean with hard reset at that winner's T and in the limit $T \to \infty$, at its trained V_{th} and β (mean over seeds), beside the clamp and the sigmoid the design study put in its place. Grey: currents that never make it fire, $I \leq V_{th}(1-\beta)$; dotted: $1/2$. (b), (c) The share of (test window, hidden unit) codes at each end, per seed, for each bounded encoder neuron (Xavier, latent mean, leaky-ReLU decoder) and winner; a LIF's code is its firing rate over T, so at $T = 19$ the high end means 19 spikes in 19 steps.

excluding zero; but the widest reaches 0.028, beyond $\Delta = 0.02$, so a cap effect of that size on EEGMMIDB is not excluded. (These runs' 30-epoch snapshots reproduce the 28 matching trainings above exactly.) None of them reached 60 epochs; the grid trains for at most 60 (Section III-F). On EEGMMIDB, where PCA errs less than the control (Section V-A), a control with $2d$ or $4d$ hidden units instead of $d+1$ erred more, by 0.06 and 0.10 (0.07 and 0.11 at 60 epochs): its validation error stopped improving by epoch 9 to 28 while its training error still fell, so under this training the narrow layers the search picks do not explain PCA's lead.

A. *Pre-validation of sequence baselines*

LSTM, GRU and Transformer autoencoders differ from the SNN-AE in recurrence, sequence length and capacity at once, so they cannot isolate what the spike costs, and their searches dominated the grid's compute. We left them out of the main run and report what they cost under one reduced budget on the development folds (FSDD fold 0 and EEGMMIDB fold 0). LSTM-AE and GRU-AE stack $L \in [1, 3]$ recurrent layers of width $H \in [8, 256]$ (floored at d), project the last state to d and decode by repeating a projection of z over time. The Transformer-AE embeds frames to $d_{\text{model}} \in [16, 128]$, adds sinusoidal positions, applies $N \in [1, 4]$ self-attention blocks (heads $\in \{1, 2, 4, 8\}$, feed-forward width $\in [32, 256]$), mean-pools to d and decodes with N further blocks and no cross-attention, so all information passes through z. All three read the window as $S = 32$ frames of 8 samples, so only the direct code is defined for them. Every family, the SNN-AE included, ran one NSGA-II of population 4 for 2 generations (12 candidates, front re-scored over 2 fresh seeds), 2 runs (seeds 42–43), with the windows, training and selection of Sections III-E–III-F, each family in its own process on one CPU thread. Their E applies the counts of Table I to every

matrix product of a cell or an attention block at each of the S positions; gate, softmax and normalization nonlinearities cost an add and a multiply each, below an exact exponential, which favours them.

Table III reports the cost of training (the fold's wall time) and of processing data after training (the measured time to reconstruct one window, and E): each of the three families' runs took at least 5.3 times the SNN-AE's wall time on the same fold, each winner took at least 3.4 times the SNN-AE's time to reconstruct a window, each winner needed at least 5.6 times the SNN-AE's energy per window (operations and memory accesses), and that energy comparison holds with every memory access priced at one SRAM size, 8 KB, 32 KB or 1 MB, with every memory price halved, with arithmetic alone, and with the first layer of a direct-coded input recomputed at every time step, as the code runs it. On quality, none of the three reached the test error of PCA on either development fold. The reduced budget can change any family's winner, and with it both its error and its per-window cost; the table describes the winners found here, not the families' best.

A Transformer whose layer normalization follows each residual sum, as ours does, can need a learning-rate warm-up to train at a fixed Adam rate; one that normalizes before each sub-layer may not, as Xiong et al. show for deep translation models [35]. In a post-hoc check we retrained two Transformer-AE winners of each development fold, 2 seeds each, with a linear warm-up over 4000 steps and, separately, with pre-norm blocks closed by a final layer normalization: on FSDD fold 0, 3 of 8 warm-up trainings and 0 of 8 pre-norm trainings left the mean-predictor plateau (control: 0 of 8), and the mean test MSE of the final fits was 0.951 with warm-up, 0.935 with pre-norm and 0.956 for the control, against 1.000 for the mean predictor (best single fit: 0.921, warm-up); on EEGMMIDB fold 0, 0 of 8 warm-up trainings and 1 of 8 pre-norm trainings left the mean-predictor plateau (control: 2 of 8), and the mean test MSE of the final fits was 0.979 with warm-up, 0.976 with pre-norm and 0.984 for the control, against 1.001 for the mean predictor (best single fit: 0.935, warm-up) (Fig. 6). Leaving the plateau means reaching 10% below the mean predictor on the monitor, which a run on FSDD can miss while still learning a little. Batch size and the shared learning rate were not varied, so the check does not show that a Transformer cannot learn this task.

VI. Results

Draft: the tables of this section are placeholders (−) until the nested-LOSO grid has run; they are regenerated from the harness's outputs.

Table V gives the test MSE and energy of every model on the headline folds, Table VI and Fig. 8 the group-level differences between the SNN-AE and each reference, Fig. 7 the same models in the quality–energy plane, Table IV the energy under each repricing, and Table VII what the SNN-AE's search selected. On energy per window, [the energy comparison has not run yet].

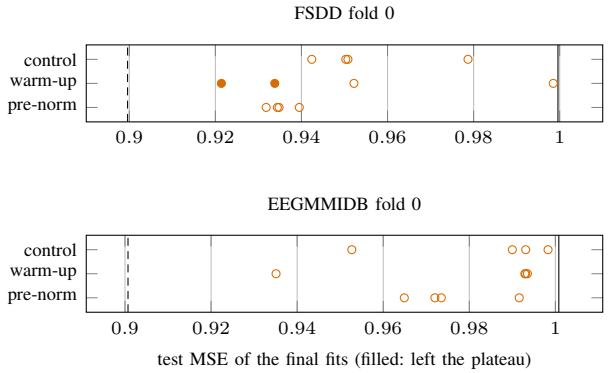


Figure 6. The Transformer-AE check: test MSE of every final fit (two winners, 2 seeds) per arm, filled when its training left the mean-predictor plateau on the monitor. Solid: the mean predictor's test MSE; dashed: 10% below it, the level that arms early stopping.

Table III
PRE-VALIDATION, DEVELOPMENT FOLDS, 2 RUNS. FOLD: WALL TIME OF THE FAMILY'S FOLD, DATA LOADING EXCLUDED (SEARCH, RE-SCORING, FINAL FIT, TEST); ×SNN: ITS RATIO TO THE SNN-AE'S, WHOSE FOLD ALSO TRAINS THE ANN-AE. TRAIN: FINAL FIT, PER TRAINING WINDOW AND EPOCH. INFER.: ONE WINDOW RECONSTRUCTED AFTER TRAINING. ONE CPU THREAD EACH.

Model	Test MSE	Fold (h)	×SNN	Train (ms/win.)	Infer. (ms/win.)	E (nJ)
FSDD						
SNN-AE	0.773 ± 0.019	0.33	1.0	0.80	0.17	2684
ANN-AE	0.713 ± 0.005	–	–	0.56	0.05	665
LSTM-AE	0.967 ± 0.026	18.1	55	15.5	4.15	170956
GRU-AE	0.916 ± 0.020	10.7	33	16.0	4.91	48431
Transformer-AE	0.957 ± 0.018	2.44	7.5	4.80	1.06	15143
PCA ($k = d$)	0.675	–	–	–	–	–
EEGMMIDB						
SNN-AE	0.425 ± 0.033	0.57	1.0	1.43	0.36	9995
ANN-AE	0.303 ± 0.001	–	–	0.93	0.08	1079
LSTM-AE	0.821 ± 0.255	16.9	30	28.5	7.58	365170
GRU-AE	0.663 ± 0.068	11.9	21	21.8	6.58	105907
Transformer-AE	0.996 ± 0.005	3.05	5.3	8.17	1.22	119327
PCA ($k = d$)	0.192	–	–	–	–	–

Reading guide. The first question is the gap between the SNN-AE and its control, per dataset and modality, and whether its interval excludes zero or lies inside $\pm\Delta$. The second is whether the SNN-AE saves energy for it: its E against the ANN-AE's, and whether that ordering survives every column of Table IV, read together with the selected encoding and T in Table VII: binary inputs turn the first layer's MACs into adds, but every step adds a read and a write of every potential. The third places both against PCA, on error and on energy; a model that does not beat PCA has not shown that its nonlinearity helps on that dataset. One rule was fixed before the grid run: where PCA errs less than both autoencoders on a modality, as it did on both development folds (Section V-A), that is reported first, and the gap between the SNN-AE and its control is then read as the price of spiking among models that both lose to a linear projection. An ordering is asserted only where the Holm-corrected p is below 0.05 and the interval excludes zero, and a finding that holds on one modality only is reported as such.

Table IV

ENERGY PER WINDOW (NJ), HEADLINE FOLDS, MEAN OVER EVERY WINNER: E (SECTION III-G), THE ENCODER ALONE, THE SHARE OF E SPENT ON MEMORY ACCESSES, E WITH EVERY ACCESS PRICED AT ONE SRAM SIZE, WITH EVERY MEMORY PRICE HALVED, ARITHMETIC ALONE, AND E AS EXECUTED (A DIRECT-CODED FIRST LAYER RECOMPUTED AT EVERY STEP). LAST ROWS OF EACH DATASET: THE SNN-AE'S ENERGY OVER EACH REFERENCE'S, PER COLUMN.

	E	Enc.	Mem.	E, one SRAM size			Mem.	Arith.	As
Model	(nJ)	only	share	8 KB	32 KB	1 MB	÷2	only	exec.
FSDD									
SNN-AE	–	–	–	–	–	–	–	–	–
ANN-AE	–	–	–	–	–	–	–	–	–
PCA	–	–	–	–	–	–	–	–	–
SNN-AE ÷ ANN-AE	–	–	–	–	–	–	–	–	–
SNN-AE ÷ PCA	–	–	–	–	–	–	–	–	–
AudioMNIST									
SNN-AE	–	–	–	–	–	–	–	–	–
ANN-AE	–	–	–	–	–	–	–	–	–
PCA	–	–	–	–	–	–	–	–	–
SNN-AE ÷ ANN-AE	–	–	–	–	–	–	–	–	–
SNN-AE ÷ PCA	–	–	–	–	–	–	–	–	–
EEGMMIDB									
SNN-AE	–	–	–	–	–	–	–	–	–
ANN-AE	–	–	–	–	–	–	–	–	–
PCA	–	–	–	–	–	–	–	–	–
SNN-AE ÷ ANN-AE	–	–	–	–	–	–	–	–	–
SNN-AE ÷ PCA	–	–	–	–	–	–	–	–	–
Siena									
SNN-AE	–	–	–	–	–	–	–	–	–
ANN-AE	–	–	–	–	–	–	–	–	–
PCA	–	–	–	–	–	–	–	–	–
SNN-AE ÷ ANN-AE	–	–	–	–	–	–	–	–	–
SNN-AE ÷ PCA	–	–	–	–	–	–	–	–	–

Table V

TEST MSE (MEAN ± STD OVER 5 SEEDS; LOWEST PER DATASET IN BOLD) AND ENERGY PER WINDOW E (NJ, MEMORY-AWARE, SECTION III-G) ON THE HEADLINE FOLDS. PCA AND MEAN ARE FITTED ONCE (NO SEEDS, NO STD); MEAN IS A STORED VECTOR AND IS NOT COSTED.

Model	FSDD MSE	E	AudioMNIST MSE	E	EEGMMIDB MSE	E	Siena MSE	E
Mean	–	–	–	–	–	–	–	–
PCA	–	–	–	–	–	–	–	–
SNN-AE	–	–	–	–	–	–	–	–
ANN-AE (control)	–	–	–	–	–	–	–	–

Table VI

SNN-AE MINUS REFERENCE, GROUP LEVEL, HEADLINE FOLDS: $\overline{d_g}$ (POSITIVE: THE SNN-AE ERRS MORE; *: HOLM-CORRECTED t-TEST $p < 0.05$; †: THE WHOLE INTERVAL LIES INSIDE ± 0.02, OVER GROUPS AND OVER FOLD MEANS) AND ITS 95% t INTERVAL.

Dataset	n_{groups}	ANN-AE	PCA	Mean
FSDD	–	–	–	–
AudioMNIST	–	–	–	–
EEGMMIDB	–	–	–	–
Siena	–	–	–	–

VII. DISCUSSION

The design study does not separate the spike's discreteness from saturation. On direct input the code sits near 0 and 1, so it is both at once; a smooth bound (the sigmoid) cost nothing and a hard one (the clamp) cost as much or more; and on Poisson input the code is not near binary, yet the gap remains. Table VII reports each end on its own, to be read beside T, since one spike moves the rate by $1/T$. Remedies that move the code off those values – a membrane read out instead of spikes in some layers, adaptive thresholds – are candidates to test. On energy, a synaptic event is not simply cheaper than a multiply–accumulate once memory is priced, so whether the SNN-AE ends up cheaper than its control depends on the encoding, T, input transform and spike rate the search selects.

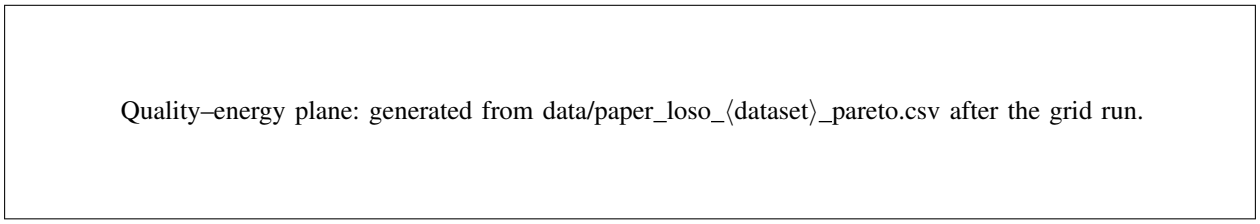


Figure 7. Test MSE (mean ± std over seeds) against energy per window E, headline folds. Down and left is better; the dashed line and the cross are PCA with $k = d$.

Table VII

SNN-AE SELECTED PER DATASET OVER EVERY (FOLD, RUN): MODAL ENCODING AND INPUT TRANSFORM (k/n AGREEING, IF NOT ALL), MEDIAN T [RANGE], MEDIAN INITIAL THRESHOLD AND DECAY, MEDIAN DEPTH L, AND THE HIDDEN NEURONS' SPIKE FRACTION ρ ON TEST WINDOWS; THEN THE TRAINED V_{th} (MEAN, AND THE LOWEST LAYER'S, WHICH NOTHING BOUNDS), THE TRAINED β, THE SHARES OF (NEURON, WINDOW) PAIRS THAT NEVER FIRED AND THAT FIRED AT EVERY STEP, AND THE FINAL NETWORK'S TRAINING ERROR. AFTER THE SLASH, THE ANN-AE CONTROL: ITS SHARE OF (HIDDEN UNIT, WINDOW) PAIRS WITH A LEAKY-RELU OUTPUT AT MOST 0.01, AND ITS TRAINING ERROR; A LEAKY RELU HAS NO UPPER END, SO IT HAS NO "ALWAYS" SHARE. READ THE SHARES BESIDE T: ONE SPIKE MOVES A RATE BY $1/T$.

Dataset	Enc.	Transf.	T	$V_{th}^{(0)}$	$\beta^{(0)}$	L	ρ	V_{th}	min	β	silent SNN / ANN	always	train SNN / ANN
FSDD	–	–	–	–	–	–	–	–	–	–	–	–	–
AudioMNIST	–	–	–	–	–	–	–	–	–	–	–	–	–
EEGMMIDB	–	–	–	–	–	–	–	–	–	–	–	–	–
Siena	–	–	–	–	–	–	–	–	–	–	–	–	–

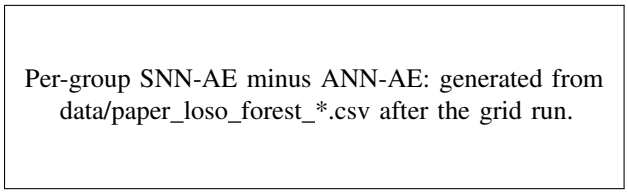


Figure 8. SNN-AE minus ANN-AE on each held-out group (dots), with the mean and its 95% t interval (bar), headline folds. The band is ± 0.02.

Table VIII

SENSITIVITY TO THE DEVELOPMENT FOLDS: SNN-AE MINUS ANN-AE, GROUP LEVEL, WITHOUT (HEADLINE) AND WITH THEM.

Dataset	Folds	n_{groups}	$\overline{d_g}$	95% CI	p_{Holm}
FSDD	–	–	–	–	–
	–	–	–	–	–
EEGMMIDB	–	–	–	–	–
	–	–	–	–	–

Per-window normalization makes silence as dense as signal for rate and latency codes, which keeps both above the break-even of Fig. 2 at every T.

Limitations. (i) *Groups.* Headline results rest on 4 of the 6 FSDD speakers, about 72 of the 109 EEGMMIDB subjects, and all of AudioMNIST's 60 and Siena's 14 groups, from at most 6 sets of models; the FSDD interval is wide by construction. (ii) *One channel, short windows.* EEG is read from one channel and every window has 256 samples; results may not transfer to multichannel or longer contexts. (iii) *Estimated, not measured energy.* E models an accelerator with local SRAMs at 45 nm and leaves out off-chip traffic, leakage and control; a real chip's consumption, and whether it can exploit sparsity, decide the final number. The repricings probe how much the conclusion depends on the memory model; they do not bound it. (iv) *Search budget.* The SNN-AE draws 48 candidates per fold and run, and the control none (it inherits the SNN-AE's widths), which favours the SNN-AE over its control; a larger budget could change the winner. (v) *Different inputs.* The ANN-AE always reads the analog window, whereas the SNN-AE may read spikes; a stochastic or latency code also loses part of the window's information, which is included in the measured gap. (vi) *Fixed latent width.* d follows prior autoencoder practice for each domain, not a sweep on these datasets. (vii) *Training cap.* At most 60 epochs. No training of the budget test reached that many (Section V); the training error in Table VII shows what the cap leaves on the headline folds. (viii) *Sequence models pre-validated, not compared.* The LSTM-, GRU- and Transformer-AE were searched with 12 candidates on two development folds only, so the conclusions concern spiking against its control, not against sequence models. (ix) *Rate read-out.* The latent is a time mean, so the timing of spikes carries nothing here; conclusions concern rate-read spiking. (x) *One window draw.* Each fold's windows are one draw, so the seeds' standard deviations leave out that sampling noise. (xi) *A first run.* A first run of the grid trained for at most 30 epochs and drew every fold's candidates from the run's seed alone, so the searches of one seed shared most of their candidates across the folds and datasets of a modality, and the cap ended every control training on the EEG folds. We found both after its results had been read, and reran the grid with the two changes of Sections III-E and III-F; the first run is released with the code.

VIII. CONCLUSION

We compared a spiking autoencoder with a non-spiking control and PCA on two speech and two EEG datasets, under

nested group-disjoint cross-validation with every architecture chosen on validation groups and design choices made on development folds. Those folds, and the fold that tests on their validation groups, are out of the headline numbers; one further fold validates on a development test group, so two datasets remain design-informed. Searched sequence autoencoders were reported on the development folds for what they cost to train and to run. We claim no cause for any gap found. The harness, profile and analysis scripts are released [36].

REFERENCES

[1] E. Lemaire, L. Cordone, A. Castagnetti, P.-E. Novac, J. Courtois, and B. Miramond, "An analytical estimation of spiking neural networks energy efficiency," in *Neural Information Processing (ICONIP 2022)*, ser. Lecture Notes in Computer Science, vol. 13623. Springer, 2023, pp. 574–587.

[2] E. O. Neftci, H. Mostafa, and F. Zenke, "Surrogate gradient learning in spiking neural networks," *IEEE Signal Processing Magazine*, vol. 36, no. 6, pp. 51–63, 2019.

[3] F. Zenke and S. Ganguli, "Superspike: Supervised learning in multilayer spiking neural networks," *Neural Computation*, vol. 30, no. 6, pp. 1514–1541, 2018.

[4] F. Zenke and T. P. Vogels, "The remarkable robustness of surrogate gradient learning for instilling complex function in spiking neural networks," *Neural Computation*, vol. 33, no. 4, pp. 899–925, 2021.

[5] I.-M. Comşa, L. Versari, T. Fischbacher, and J. Alakuijala, "Spiking autoencoders with temporal coding," *Frontiers in Neuroscience*, vol. 15, p. 712667, 2021.

[6] H. Kamata, Y. Mukuta, and T. Harada, "Fully spiking variational autoencoder," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, no. 6, 2022, pp. 7059–7067.

[7] G. C. Cawley and N. L. C. Talbot, "On over-fitting in model selection and subsequent selection bias in performance evaluation," *Journal of Machine Learning Research*, vol. 11, pp. 2079–2107, 2010.

[8] M. Horowitz, "Computing's energy problem (and what we can do about it)," in *IEEE International Solid-State Circuits Conference (ISSCC) Digest of Technical Papers*, 2014, pp. 10–14.

[9] J. K. Eshraghian *et al.*, "Training spiking neural networks using lessons from deep learning," *Proceedings of the IEEE*, vol. 111, no. 9, pp. 1016–1054, 2023.

[10] P. J. Werbos, "Backpropagation through time: What it does and how to do it," *Proceedings of the IEEE*, vol. 78, no. 10, pp. 1550–1560, 1990.

[11] S. Thorpe, A. Delorme, and R. Van Rullen, "Spike-based strategies for rapid processing," *Neural Networks*, vol. 14, no. 6–7, pp. 715–725, 2001.

[12] I. M. Comsa, K. Potempa, L. Versari, T. Fischbacher, A. Gesmundo, and J. Alakuijala, "Temporal coding in spiking neural networks with alpha synaptic function," in *ICASSP 2020 – 2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2020, pp. 8529–8533.

[13] Z. Zhou, Y. Zhu, C. He, Y. Wang, S. Yan, Y. Tian, and L. Yuan, "Spikformer: When spiking neural network meets transformer," in *International Conference on Learning Representations (ICLR)*, 2023.

[14] M. Dampfhoffer, T. Mesquida, A. Valentian, and L. Anghel, "Are SNNs really more energy-efficient than ANNs? an in-depth hardware-aware study," *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 7, no. 3, pp. 731–741, 2023.

[15] Z. Yan, Z. Bai, K. Tang, and W.-F. Wong, "Reconsidering the energy efficiency of spiking neural networks inference from analytical perspectives," arXiv:2409.08290, 2024.

[16] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[17] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," in *Advances in Neural Information Processing Systems*, vol. 27, 2014, pp. 3104–3112.

[18] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder–decoder for statistical machine translation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1724–1734.

[19] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," *arXiv preprint arXiv:1412.3555*, 2014.

[20] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998–6008.

[21] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," *arXiv preprint arXiv:1607.06450*, 2016.

[22] W. Gerstner and W. M. Kistler, *Spiking Neuron Models: Single Neurons, Populations, Plasticity*. Cambridge University Press, 2002.

[23] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002.

[24] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, 2015.

[25] L. Prechelt, "Early stopping – but when?" in *Neural Networks: Tricks of the Trade*, ser. Lecture Notes in Computer Science. Springer, 1998, vol. 1524, pp. 55–69.

[26] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.

[27] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, ser. PMLR, vol. 9, 2010, pp. 249–256.

[28] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2015, pp. 1026–1034.

[29] Z. Jackson *et al.*, "Free spoken digit dataset," GitHub repository, 2018. [Online]. Available: https://github.com/Jakobovski/free-spoken-digit-dataset

[30] S. Becker, J. Vielhaben, M. Ackermann, K.-R. Müller, S. Lapuschkin, and W. Samek, "AudioMNIST: Exploring explainable artificial intelligence for audio analysis on a simple benchmark," *Journal of the Franklin Institute*, vol. 361, no. 1, pp. 418–428, 2024.

[31] G. Schalk, D. J. McFarland, T. Hinterberger, N. Birbaumer, and J. R. Wolpaw, "BCI2000: A general-purpose brain-computer interface (BCI) system," *IEEE Transactions on Biomedical Engineering*, vol. 51, no. 6, pp. 1034–1043, 2004.

[32] A. L. Goldberger, L. A. N. Amaral, L. Glass, J. M. Hausdorff, P. C. Ivanov, R. G. Mark, J. E. Mietus, G. B. Moody, C.-K. Peng, and H. E. Stanley, "PhysioBank, PhysioToolkit, and PhysioNet: Components of a new research resource for complex physiologic signals," *Circulation*, vol. 101, no. 23, pp. e215–e220, 2000.

[33] P. Detti, G. Vatti, and G. Zabalo Manrique de Lara, "EEG synchronization analysis for seizure prediction: A study on data of noninvasive recordings," *Processes*, vol. 8, no. 7, p. 846, 2020.

[34] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[35] R. Xiong, Y. Yang, D. He, K. Zheng, S. Zheng, C. Xing, H. Zhang, Y. Lan, L. Wang, and T. Liu, "On layer normalization in the Transformer architecture," in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 119, 2020, pp. 10 524–10 533.

[36] Anonymous, "Reference suppressed for double-blind review," 2026, code repository citation withheld; to be restored in the camera-ready version.